

← CT Scan Explorer Build Guide :).docx

the full build guide · @hinumpy

Build a 3D CT scan explorer with one prompt

CT scans are flat slices with the sides flipped, which makes them really hard to learn to read. This guide walks you through building a tool that turns one real hospital CT scan into 3D organs and syncs every view, so you click an organ and see it everywhere.

You will download a free dataset, install Codex, paste one prompt, and let GPT-6 Astra build the whole thing. Then you check that it is actually correct, because that part is on you.

Heads up: this is a learning project. It is not a medical device and should never be used for diagnosis.

At a glance

What you get	A web app with 8 abdominal structures in 3D, synced axial, coronal, and sagittal views, and click-to-find for every organ
Build time	About 30 minutes of Astra working (my run took 26m 54s), plus a 3 GB download
Difficulty	Beginner friendly. You mostly copy, paste, and approve
Stack	Python (nibabel, scikit-image, trimesh) · Next.js · React Three Fiber · Zustand
Data	TotalSegmentator CT dataset, small subset (102 scans, free, CC BY 4.0)

What you need

- **A Mac.** Every command here is written for macOS. Linux is nearly identical.
- **A ChatGPT plan with GPT-6 Astra in Codex.** Plus includes limited Astra usage in Codex, so check your plan and start with a fresh usage window.
- **Node.js and npm.** Check with `node --version`.
- **Python 3.** I built this with Python 3.13.
- **About 7 GB of free space** while you unzip. You can delete the zip afterward.

Step 1: Download the dataset

Open Terminal and paste these one at a time. The download is about 3 GB.

```
mkdir -p ~/ct-explorer/data/raw
cd ~/ct-explorer/data/raw
curl -L -o small.zip
"https://zenodo.org/records/10047263/files/Totalsegmentator_dataset_small_v201.zip?"
```

← CT Scan Explorer Build Guide :).docx

Check the md5 line. It should end in `6b5524af4b15e6ba06ef2d700c0c73e0`. If it does not match, the download broke, so delete `small.zip` and run the curl line again.

Check the folders. `ls` should show `meta.csv` plus folders like `s0011` and `s0058`. Then free up the space:

```
rm ~/ct-explorer/data/raw/small.zip
```

Step 2: Install Codex

```
npm install -g @openai/codex
codex --version
```

You need **version 0.153.0 or newer** for Astra. The package name is `@openai/codex`. The plain `codex` package on npm is an unrelated project.

Hit an error? Jump to the troubleshooting table at the end. The two most common ones are `EACCES` and `command not found`.

Step 3: Open Codex in your project folder

```
cd ~/ct-explorer
codex
```

1. **Sign in with ChatGPT** when the browser opens, so it uses your plan.
2. **Type `/model` and pick GPT-6 Astra.** High reasoning is enough.
3. **Check the header** says the directory is `~/ct-explorer`.

Important: run Codex locally in this folder. A cloud workspace cannot see the files on your laptop, and the prompt will stop and tell you the data folder does not exist.

Step 4: Paste the prompt

This is the full prompt from the reel. Copy all of it, paste it into Codex, and press Enter. A long paste may show up collapsed, which is normal.

```
build a complete, working web app end to end without stopping to ask me questions. make
reasonable decisions, verify your own work, and report back when it's done.

### what we're building
an interactive ct anatomy explorer for students: 3d abdominal organs built from a real ct
scan, plus axial, coronal, and sagittal slice views that all stay synced to one crosshair.
click an organ anywhere and every view jumps to it.

### data (already downloaded)
./data/raw contains the totalsegmentator ct small subset. each subject folder (like s0011)
should have ct.nii.gz and a segmentations/ folder with one .nii.gz mask per structure. verify
this first. if data/raw is empty or structured differently, stop and tell me.

### step 1: environment
create a python venv in ./pipeline and install nibabel numpy scikit-image trimesh fast-
simplification pillow. scaffold ./web with next.js (app router, typescript), @react-
three/fiber, @react-three/drei, zustand.

### step 2: pick a scan (pipeline/pick_subject.py)
load one subject at a time and reorient with nib.as_closest_canonical. a subject qualifies
only if these 7 masks are non-empty: liver, spleen, stomach, kidney_left, kidney_right,
```

← CT Scan Explorer Build Guide :).docx

1. reorient ct and masks with nib.as_closest_canonical so axes are i -> patient right, j -> anterior, k -> superior (RAS+). read spacing from the header, never assume it.
2. uint8 label volume: 1 liver, 2 spleen, 3 stomach, 4 kidney_right, 5 kidney_left, 6 aorta, 7 inferior_vena_cava, 8 spine (union of all vertebrae_* masks).
3. crop ct and labels to the bounding box of labels > 0 plus 20 mm margin, clamped.
4. window ct at level 50 / width 400: clip HU to [-150, 250], scale to 0-255 uint8.
5. write web/public/data/ct.bin and labels.bin as raw uint8, numpy C order, shape (ni, nj, nk). write meta.json with shape, spacing_mm, crop offset, volume center in mm, and per label: id, name, centroid (i, j, k).
6. per label: skimage marching_cubes with spacing, light taubin smoothing, decimate to max ~40k faces. verts are (r, a, s) mm. convert to gltf coords centered on volume center c: $x = (r - c_r)$, $y = (s - c_s)$, $z = (a - c_a)$. run fix_normals. export web/public/data/meshes/<name>.glb.

step 4: web app layout + 3d

- desktop: organ sidebar on the left, then a 2x2 grid: 3d top-left, axial top-right, coronal bottom-left, sagittal bottom-right. mobile: organ chips in a horizontal row, panels stacked.
- one zustand store: crosshair {i, j, k} in cropped voxel indices, selectedOrgan, visibleOrgans.
- 3d: load all glbs, selected organ opaque, others ~35% opacity. orbit controls. camera starts on +z looking at origin so patient right is on screen left.
- three translucent planes at the crosshair (axial at k, coronal at j, sagittal at i) using the same transform: $x = -(i*s_i - c_r)$, $y = k*s_k - c_s$, $z = j*s_j - c_a$.
- clicking a mesh sets the crosshair to the clicked point (inverse transform, rounded, clamped) and selects the organ. clicking a sidebar name selects it and moves the crosshair to its centroid.

step 5: 2d panels (canvas)

fetch ct.bin and labels.bin once. index = $i*nj*nk + j*nk + k$. follow this display convention exactly:

- axial (fixed k): column = $ni-1-i$, row = $nj-1-j$. R left, L right, A top, P bottom. caption "viewed from the feet"
- coronal (fixed j): column = $ni-1-i$, row = $nk-1-k$. R left, L right, S top, I bottom. caption "facing the patient"
- sagittal (fixed i): column = $nj-1-j$, row = $nk-1-k$. A left, P right, S top, I bottom. caption "viewed from the patient's left"

scale by voxel spacing. ct pixels stay pure grayscale. overlays: selected organ outline in its color, faint fill for visible organs, crosshair lines with a small circle, 50 mm scale bar, "W 400 / L 50" in a corner. click or drag sets the in-plane crosshair coords, scroll wheel or a slider moves the slice. everything stays synced.

step 6: design

- dark plum theme (radiology is read in dim rooms): background #1A0D14, panels #2A1520, borders #3A1C2B, accent #E8447A, text #FCE4EC, secondary text #C9A9B4
- soft pastel organ colors readable on plum: liver dusty rose, spleen lavender, stomach peach, kidneys two different corals, aorta red-pink, ivc periwinkle, spine cream
- fonts: poppins for ui, eb garamond for the large selected-organ name over the 3d view. sentence case, no all-caps labels
- one bold moment only: selecting an organ shows its name large in eb garamond and fades it to full opacity
- footer: "for learning only, not for diagnosis" and "ct data: totalsegmentator dataset (wasserthal et al.), university hospital basel, cc by 4.0" linking to <https://zenodo.org/records/10047292>
- loading state while data loads, clear error if a file fails, respects reduced motion, works in iphone safari

verification (all required before you report done)

1. assert in python that the liver centroid i is greater than the spleen centroid i (liver is on the patient's right). if it fails, fix orientation.
2. save axial, coronal, sagittal pngs through the liver centroid using the display convention above. look at them and confirm: liver on screen left in axial and coronal, spine at the bottom of axial, spine on the right in sagittal.
3. npm run build passes with no errors.
4. run the dev server, screenshot at desktop and mobile widths. confirm meshes render, planes line up with the meshes, and selecting kidney_left puts the crosshair inside the kidney

← CT Scan Explorer Build Guide :).docx

- subject id used
- exact commands to run it
- every bug you hit and how you fixed it
- anything you could not verify

Step 5: While it builds

- **Approve** installs and scripts inside `ct-explorer`: `pip install`, `npm install`, `create-next-app`, the Python scripts, and `npm run build`.
- **Decline** anything with `sudo` or anything that touches files outside the project folder.
- **If an install fails the first time**, Codex may ask to run it again. Approve it. This happened in my build and it kept going.
- **Don't interrupt it**. Only reply if it asks you something.
- **It may shrink the scan** to keep the web files small. Mine was resampled to about 2 mm voxels, which is expected.

By the end of my run, Codex warned I had less than 10% of my 5-hour usage limit left. Plan for that before you start.

Step 6: Open your app

```
cd ~/ct-explorer/web
npm run dev
```

Open **`http://localhost:3000`** in your browser and keep that terminal tab open. For a smoother version with no dev badge, use `npm run build` and then `npm start`.

Step 7: Check it is actually right

AI can build something that looks great and is anatomically wrong. Run these three checks before you show anyone.

1. **Axial panel:** the liver (the biggest organ) is on the screen's left, and the spine is at the bottom.
2. **Click the left kidney in 3D:** the crosshair lands inside the kidney in all three panels.
3. **Scroll the axial slider to kidney level:** both kidneys show with the spine between them, and the left kidney is on the screen's right.

Why left is on the right: CT slices are displayed as if you are standing at the patient's feet looking up. The patient's left ends up on your right. If your app shows the liver on the right side, the orientation is flipped, so tell Codex which check failed.

Troubleshooting

Problem	Fix
npm ERR! code EACCES	Some npm folders are owned by the admin account. Run <code>sudo chown -R \$(id -u):\$(id -g) ~/.npm ~/.npm-global</code> , then install again without sudo.

← CT Scan Explorer Build Guide :).docx

codex	>> ~/ .Zshrc, then source ~/ .Zshrc and try again.
Codex says data/raw does not exist	Codex is running in the wrong place. Quit, <code>cd ~/ct-explorer</code> , and start <code>codex</code> from there.
Conda errors every time Terminal opens	Safe to ignore. The build uses its own Python environment.
Port 3000 is busy	Use whatever address the terminal prints instead, like localhost:3001.
The liver is on the right side	Orientation is flipped. Tell Codex that check 1 in Step 7 failed and ask it to fix the display convention.

Make it resume ready

Before you push to GitHub

- **Add a .gitignore** with `data/`, `pipeline/.venv/`, `web/node_modules/`, and `web/.next/`. Never push the 3 GB of raw scans.
- **Put a GIF at the top of your README** of you clicking an organ and every view jumping to it.
- **Credit the dataset** in your README and your app footer. CC BY 4.0 requires attribution.
- **Deploying on Vercel?** Set the root directory to `web`.

Resume bullet template

Fill in your own real numbers. Do not make them up.

"Built an interactive CT anatomy explorer from a clinical CT scan (TotalSegmentator dataset), generating 3D meshes for [n] structures from segmentation masks and syncing them with axial, coronal, and sagittal views in Next.js and React Three Fiber; added automated orientation checks and kept web assets under [x] MB."

Be ready to explain it

If an AI agent wrote most of the code, say that. What makes you stand out is being able to explain what you checked: why the patient's left shows on the screen's right, how the organ masks become 3D models, and how one crosshair keeps every view in sync.

Credits and license

Data: TotalSegmentator dataset (Wasserthal et al.), University Hospital Basel, CC BY 4.0. Full dataset: zenodo.org/records/10047292. Small subset used here: zenodo.org/records/10047263.

For learning only. Not a medical device and not for diagnosis.

go build something <3

made by @hinumpy · hinna

@hinumpy ·